

Taller: Control de Calidad de Software

Unidad 1 - Clase 2 - Ciclo de vida y Arquitecturas





- Ciclo de vida de un proyecto
- Metodologías de desarrollo
 - Cascada
 - Modelo espiral
 - Prototipo
 - Desarrollo de software ágil
- Arquitectura de aplicaciones



- **Cascada ('80)**
- **Modelo espiral ('90)**
- **Modelo prototipo ('80 / '90)**
- **Desarrollo de software ágil (+2000)**



Es un enfoque lineal y secuencial en el que el desarrollo avanza a través de etapas predefinidas como requisitos, análisis, diseño, codificación, pruebas y mantenimiento. Cada etapa debe completarse antes de pasar a la siguiente.

- Fácil comprensión, planificación y seguimiento.
- Alta calidad del producto resultante.
- Permite trabajar con personal poco cualificado.

- Todos los requisitos deben estar definidos desde el principio.
- Si hay errores es difícil volver atrás en las etapas del desarrollo del software.
- Se debe finalizar el software completo para que esté disponible

Modelo en espiral (Spiral Model)



Se basa en la hipótesis de que el desarrollo de software es un ciclo iterativo que se repite hasta alcanzar los objetivos establecidos. Tiene la capacidad de manejar la gran cantidad de riesgos que pudieran ocurrir al desarrollar cualquier software.

- Gestión de riesgo
- Flexibilidad
- Mejora continua
- Mayor participación de los involucrados

- Complejidad: las iteraciones complejizan el desarrollo
- Costo y tiempo: es más costoso que el de cascada por las iteraciones
- Requiere experiencia en identificación y gestión de riesgos
- Mayor participación de los involucrados
- No es adecuado para proyectos de poca envergadura



Se utiliza para reunir requisitos de manera rápida y temprana. Se crean prototipos del software para que los usuarios y los desarrolladores puedan entender cómo debería ser el producto final y realizar ajustes según sea necesario.

- Retroalimentación temprana: El usuario puede ver rápidamente una primera versión del software
 - Clarificación de requisitos
 - Reducción de costos
 - Mejora la calidad
-
- Posible falta de rigor
 - Complejidad de mantenimiento, debido a la evolución del prototipo a la versión final.
 - Aumento de costos a largo plazo si no se planifica adecuadamente.



Es un enfoque basado en valores y principios que fomenta la entrega continua de software funcional, la colaboración entre equipos multidisciplinarios y la capacidad de adaptación a los cambios. Scrum y Kanban son dos de los marcos de trabajo ágil más populares.

- Mayor satisfacción del cliente: el cliente está involucrado en todo el proceso de desarrollo
- Mayor flexibilidad: Responden rápidamente a cambios de alcance y prioridades del proyecto
- Entregas Continuas, mejor calidad de software

- Experiencia de todo el equipo involucrado
- Complejidad en la gestión de proyecto
- Resistencia al cambio
- Falta de documentación



ROLES

Product Owner / SCRUM Master / Equipo SCRUM

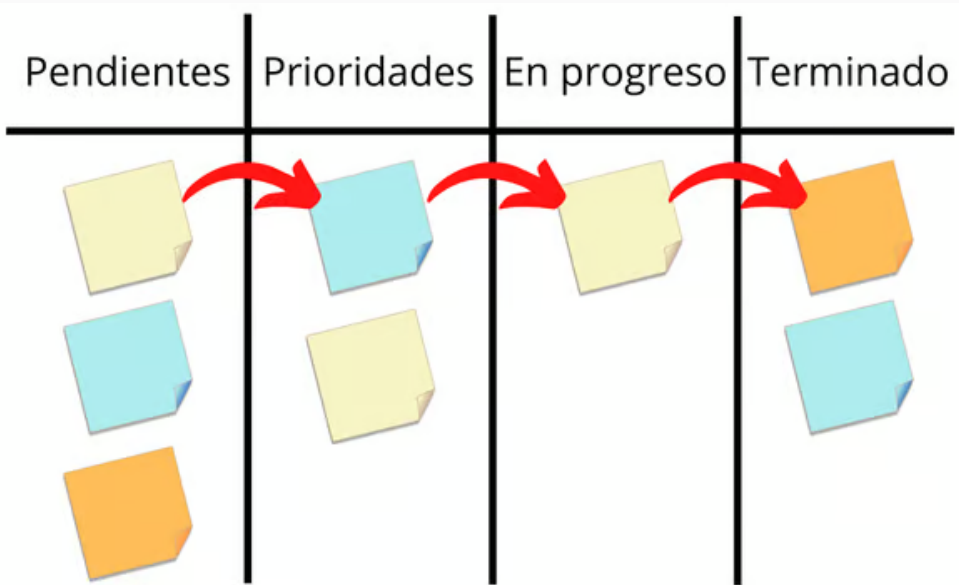
ARTEFACTOS

Product Backlog / Sprint Backlog / Incremento

EVENTOS

Planificación / Revisión / Retrospectiva

Scrum se enfoca en la entrega continua de valor al cliente, la transparencia, la adaptación al cambio y el trabajo en equipo colaborativo. Es utilizado por muchas organizaciones debido a su capacidad para gestionar proyectos complejos de manera efectiva y para responder rápidamente a los cambios en los requisitos y en el entorno.



La palabra Kanban es de origen japonés y su significado está vinculado al concepto de “justo a tiempo”. En la práctica, el método Kanban se organiza en un tablero o tabla (tablero Kanban), dividido en columnas, que muestra cada flujo dentro del proyecto de producción de software. A medida que el desarrollo evoluciona, la información contenida en la tabla cambia, y cada vez que entra en juego una nueva tarea, se crea una nueva “tarjeta”.



¿Que es la arquitectura de Software?

La arquitectura de aplicaciones de software se refiere a la estructura fundamental de un sistema de software, que incluye sus componentes, sus relaciones, y cómo esos componentes interactúan entre sí para lograr los objetivos del sistema. Una arquitectura bien diseñada proporciona una base sólida para el desarrollo, la implementación y el mantenimiento de la aplicación.

Componentes

Conectores

Modelo de datos

Patrones de diseño

Capas

MVC

Servicios

Escalabilidad

1990

Cliente servidor

Monolíticos

3 Capas

Desarrollo orientado a obj.

Tecnologías emergentes



2024

Microservicios

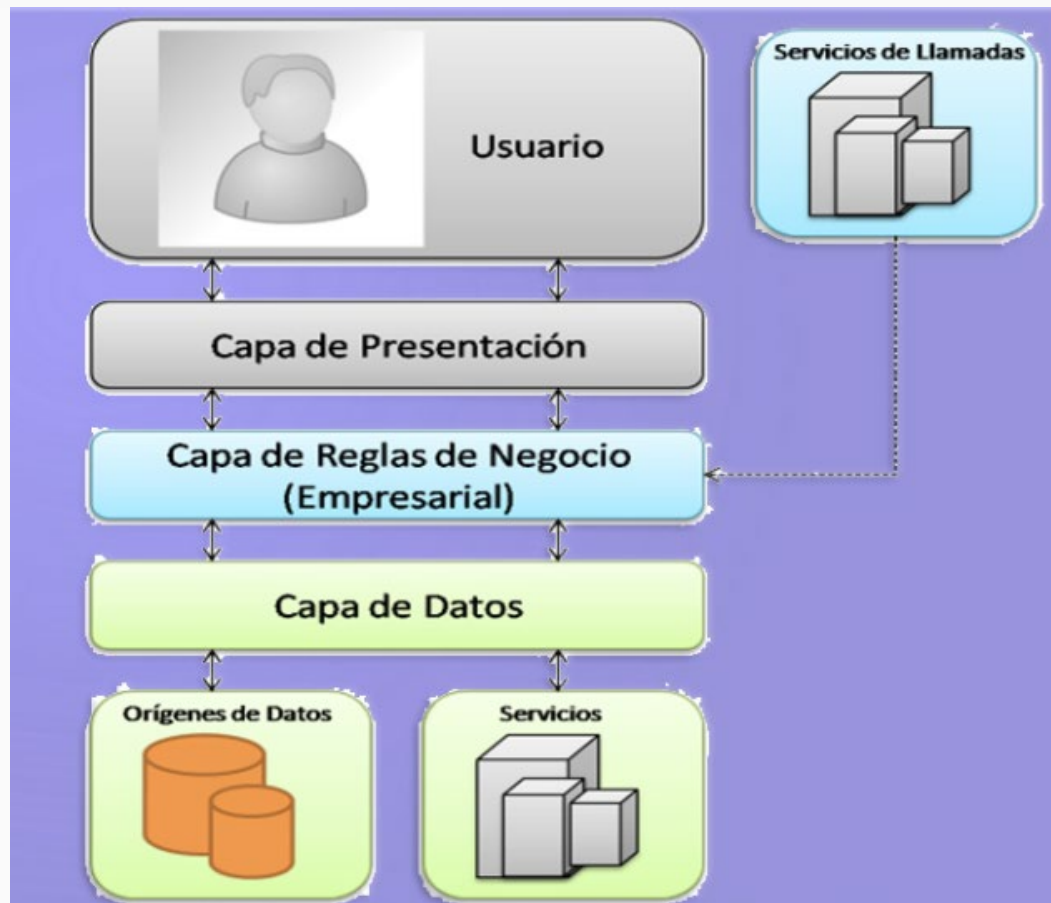
Contenedores

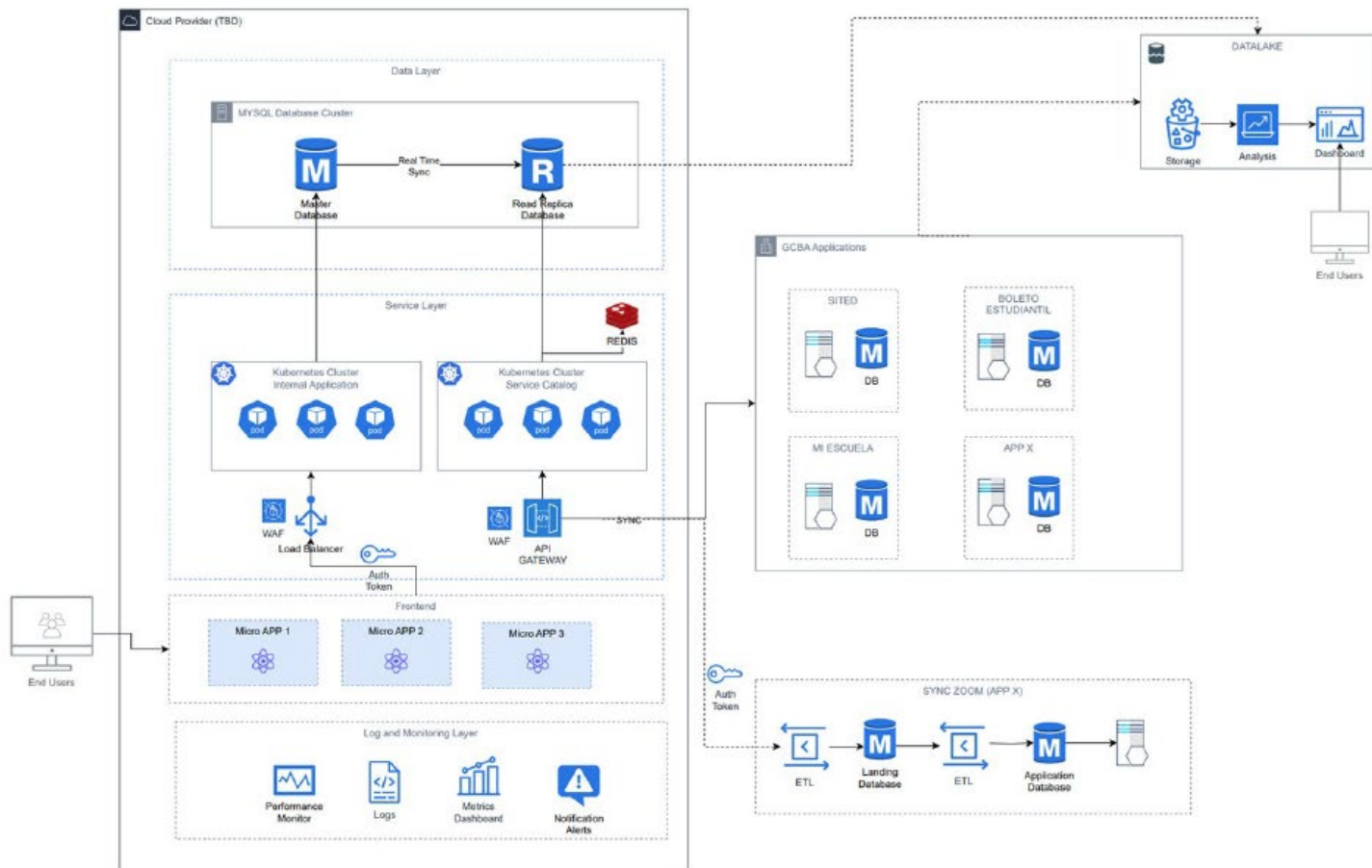
Computación en la nube

APIs y microservicios rest

Big Data

Seguridad centrada en la
arquitectura







¿Preguntas?



GRACiAS

